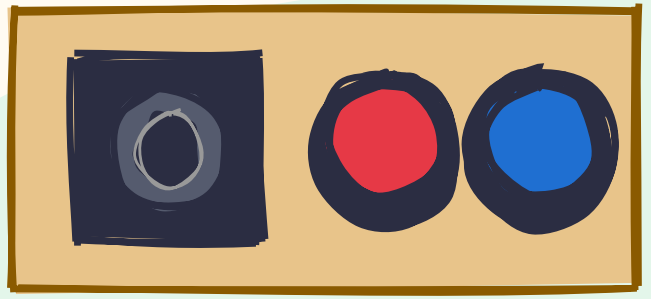
Young Innovators · Class 32

Build Your Own Game Controller

Your hardware. Your game. A player you design for.

Joystick + buttons | Serial → Pygame | Accessible design



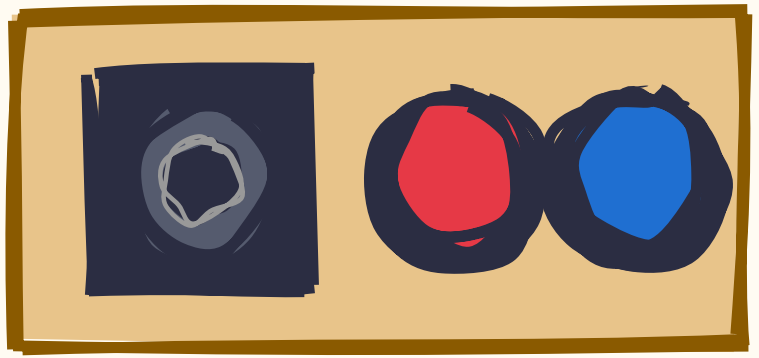
Did you know?
The Xbox Adaptive Controller was designed with disabled gamers, so players can plug in big buttons, foot pedals or mouth switches.

Let's think...

What if a normal controller doesn't fit you?

One size doesn't fit all

A standard controller needs two hands, ten fingers and quick thumbs. What about a player with one hand, weak grip, or who uses their feet? Big companies are learning to design for them. Today, so do you.



Use a phone?

Too bad!

Build one!

Today: you build a **custom controller** with Arduino, connect it to a **Pygame** game, and design it for a **real player**.

Learning objectives

By the end of this class I can...

1	**Read the inputs:** read a **joystick** (analog, with a **dead zone**) and **buttons** (INPUT_PULLUP)	**Proof:** Serial shows x, y and button values that change as I move
2	**Stream to Pygame:** send inputs over **serial** and use them in a **Pygame** game with no lag	**Proof:** my character moves smoothly with my controller
3	**Design for a player:** design an **accessible** controller for a specific player and **playtest** it with them	**Proof:** a classmate used my controller in a way a normal one wouldn't allow

Objective 3 · Design for a player · video

Video time!



Introducing the Xbox Adaptive Controller

XBOX · about 3 minutes · youtu.be/9fcK19CAjWM

Watch for...

1. Who was the controller **designed for**?
2. How can players **plug in** their own buttons and switches?
3. Who did Microsoft **work with** to design it?

Scan to watch

After the video: Name one body movement, other than thumbs, that could control a game.

Objective 1 · Read the inputs

Analog sticks and digital buttons

The idea

A **joystick** is two potentiometers: X and Y. analogRead gives **0 to 1023**, about **512** in the middle.
Real sticks never rest exactly at 512, so we use a **dead zone**: ignore small values near the centre so the character doesn't drift.

Buttons with **INPUT_PULLUP**: no extra resistor; read **HIGH** when free, **LOW** when pressed. So pressed = **!digitalRead(pin)**.

Example: a car steering wheel



x: 0 … 512 … 1023

dead zone ±10%



pressed = LOW

Turning the steering wheel a tiny bit does nothing; there's a little 'play'. That dead zone stops the car twitching from every tiny wobble.

Objective 2 · Stream to Pygame

Streaming inputs without lag

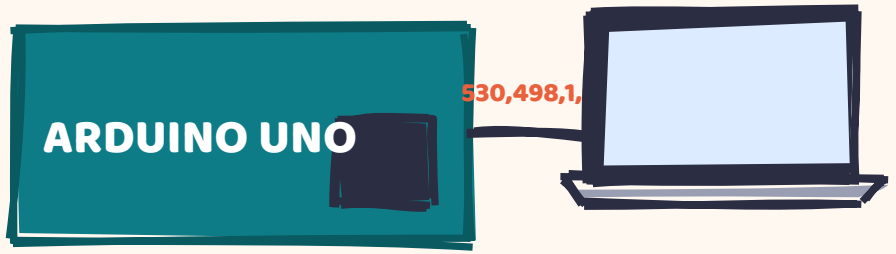
The idea

The Arduino sends one line every 20 ms: **x,y,a,b** e.g. **530,498,1,0**

Pygame runs at 60 FPS. Each frame it reads **everything waiting** in the serial buffer and uses only the **newest complete line**. If you read just one old line per frame, the buffer fills up and the game lags behind your hand.

Use a fast baud rate: **115200**.

Example: reading the latest news



If you read one newspaper a day but 3 arrive daily, you fall further behind each day. Skip to today's paper! Games do the same with input data.

Objective 3 · Design for a player

Accessible design starts with a person

The idea

Don't design for 'everyone'. Design with **one real player**, then it often works better for many.

Ask: Which body movements are **comfortable**? How much **force**? How **fast**? One hand or two? Seated?

Ideas: **big** buttons, **foot** pedals, a **one-hand** layout, **tilt** control, **hold** instead of tap, adjustable **sensitivity**.

Example: kerb ramps



foot pedal

Ramps at road crossings were made for wheelchairs, but parents with prams, delivery workers and travellers with suitcases use them all day. Designing for one need helps many.

LEARN

Real world

Controllers for everyone



Xbox Adaptive Controller
Plug in any switch, pedal or big button



AbleGamers
A charity that builds custom setups for disabled players



Sony Access controller
Customisable layout for PlayStation



Rehab games
Physiotherapy turned into games with custom inputs



Who do you know who might find a normal controller hard to use? What would help them?

Designed with players
These controllers were tested with disabled gamers at every stage, not just at the end.

Quick quiz

Controller check

Question	Answer
1. Joystick centre reads about…?	
2. Why a dead zone?	
3. INPUT_PULLUP button reads what when pressed?	
4. Why use only the newest serial line?	
5. Best first step in accessible design?	

Answers

Cover! 1: 512 · 2: stop drift from tiny wobbles · 3: LOW · 4: avoid lag from old data piling up · 5: talk to the player who will use it

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Controller challenge
2. Our controller kit
3. Arduino: stream the inputs
4. Pygame: use the controller
5. Design with your player
6. Playtest record
7. Inclusive designer challenges

Class game

Controller challenge

How to play

1. Pairs. One player uses a normal keyboard to play a simple browser game.
2. Round 2: play with ONE hand only. Round 3: with your elbow. Round 4: with your foot on a big book 'pedal'.
3. Partner notes: what was hard? What would have helped?
4. Share the top 3 problems with the class.

Then discuss

Which change would make the biggest difference: bigger buttons, fewer buttons, or a different body part?



Big idea

Feeling the problem helps you design the solution.

Build it

Our controller kit

Arduino Uno
Reads all inputs

Joystick module
VRx → A0, VRy → A1, 5V, GND

2+ big arcade buttons
D2 and D3 to GND (INPUT_PULLUP)

Box / card / 3D print
Controller body

Pygame + pyserial
Game side

A player
Your design partner

Optional inputs: a foot switch, a tilt sensor (MPU6050), or a pressure pad (two foil sheets + foam).

Comfort
Rounded edges, no loose wires. Test with your player for short sessions and stop if anything hurts.

Objective 1 · Read the inputs · Build step 1 of 3

Arduino: stream the inputs

```
void setup() {
  Serial.begin(115200);
  pinMode(2, INPUT_PULLUP);       // button A
  pinMode(3, INPUT_PULLUP);       // button B
}

void loop() {
  Serial.print(analogRead(A0));  Serial.print(',');   // x
  Serial.print(analogRead(A1));  Serial.print(',');   // y
  Serial.print(!digitalRead(2)); Serial.print(',');   // A: 1 = pressed
  Serial.println(!digitalRead(3));                    // B
  delay(20);                                          // 50 times a second
}
```

Test

Serial Monitor at 115200. Move the stick: numbers change? Press: 0 → 1?

Centre

Note your stick's real resting values. Yours might be 507,519.

Then close

Close Serial Monitor before running Python.

MAKE

Objective 2 · Stream to Pygame · Build step 2 of 3

Pygame: use the controller

```
import pygame, serial
ser = serial.Serial("COM3", 115200, timeout=0)        # your port
pygame.init()
screen = pygame.display.set_mode((600, 400)); clock = pygame.time.Clock()
x, y, jx, jy, a, b, buf = 300, 200, 512, 512, 0, 0, ""

running = True
while running:
    for e in pygame.event.get():
        if e.type == pygame.QUIT: running = False

    buf += ser.read(ser.in_waiting or 1).decode(errors="ignore")
    lines = buf.split("\n"); buf = lines[-1]          # keep unfinished part
    if len(lines) > 1:
        try: jx, jy, a, b = map(int, lines[-2].split(","))   # NEWEST line
        except ValueError: pass

    dx, dy = (jx - 512) / 512, (jy - 512) / 512
    if abs(dx) < 0.1: dx = 0                            # dead zone
    if abs(dy) < 0.1: dy = 0
    x, y = x + dx * 6, y + dy * 6

    screen.fill((20, 22, 40))
    col = (255, 80, 80) if a else (255, 183, 3)
    pygame.draw.circle(screen, col, (int(x), int(y)), 20 + 10 * b)
    pygame.display.flip(); clock.tick(60)
pygame.quit()
```

Newest

lines[-2] is the latest COMPLETE line; lines[-1] may be half-arrived.

Dead zone

Below 10% tilt we treat as zero: no drifting.

Upgrade

Swap the circle for your Catch the Stars game from YI-19!

MAKE

Objective 3 · Build step 3 of 3

Design with your player

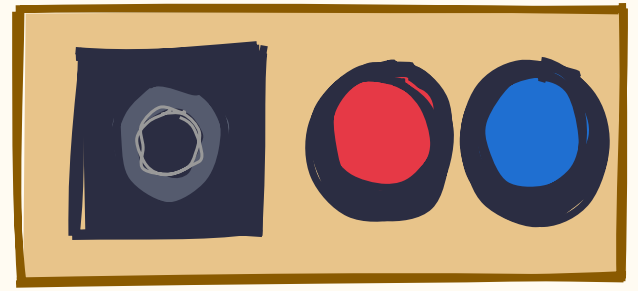
a Interview your player (a classmate who chooses a 'challenge': one hand, feet only, low strength...).

b Sketch 2 controller layouts. Let them choose.

c Build the body: big buttons where THEY can reach, joystick or tilt if it suits them.

d Add **sensitivity**: a key in the game to make movement slower or faster.

e Playtest 3 times. After each test, change ONE thing they asked for.



Check: Your player can play your game comfortably with your controller.

Never pressure someone to share personal needs. Let them choose a role-play challenge if they prefer.

Test like an engineer

Playtest record

Test	Result	Notes
Moves with no drift at rest		
Buttons respond instantly		
Player's comfort (1 to 5)		
Change after test 1		
Change after test 2		

Debug checklist

Character drifts? Increase the dead zone or use your real centre values. Laggy? Use 115200 baud and read all waiting data each frame. ValueError? Half lines: the try/except skips them.

Listen to your player more than your own ideas. They are the expert on their body.

MAKE

Level up!

Inclusive designer challenges

Choose your challenge

Remap: let the player choose which input does which action, saved in a settings file.

Hold mode: replace rapid tapping with 'hold to keep doing it'.

Tilt: use an MPU6050 so tilting the controller steers.

USB HID: use an Arduino Leonardo or Pro Micro to act as a real keyboard for ANY game.

Think like an engineer

Why is it better to let players remap controls than to pick 'the best' layout for them?

Boss level

Make your controller work with a browser game by turning it into a USB keyboard (Leonardo).

Objective check

Can I really do it? Prove it.

01	Show Serial: x, y and buttons. What's your stick's resting value and dead zone?	○ ○ ○
02	Show your Pygame game responding instantly. Why read the newest line?	○ ○ ○
03	Who did you design for? Show one change you made after playtesting.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Expo: each team demos its controller with its player. Awards for most inclusive, most creative and best-built.

Show what you know

What did we learn?

- [x] Joysticks: two potentiometers, 0 to 1023
- [x] Dead zones stop drift; INPUT_PULLUP buttons read LOW when pressed
- [x] Stream x,y,a,b lines; always use the newest
- [x] Accessible design starts with a real person
- [x] Playtest, listen, change one thing, repeat

Exit ticket: why does the game use lines[-2] instead of lines[0]?

At home

Look at a remote control at home. How could it be redesigned for your grandparent?



Next class:
Phase 2 complete!
Capstone time!